

Commutative m -ary Quasigroup Actions in a 2-adic Regime: A CSIDH-like Commutative-Action Candidate and a Key Exchange Protocol

Danilo Gligoroski `danilog@ntnu.no`
Norwegian University of Science and Technology (NTNU)

2 May 2026

Keywords: commutative actions, m -ary quasigroups, 2-adic structure, key exchange, CSIDH analogy, polynomial permutations

Abstract: Isogeny-based cryptography has popularized commutative-action key exchange (e.g., CSIDH-style class-group actions) as a post-quantum analogue of Diffie–Hellman.[DF17, CLM⁺18] We propose *Commutative Quasigroup Actions (CQA)*: a fully general m -ary ($m \geq 3$) quasigroup-inspired commutative action over $\mathbb{Z}/2^w\mathbb{Z}$ whose commutativity holds in a *2-adic regime* motivated by polynomial permutations on the units of $\mathbb{Z}_{2^w}$. [MŠG10] The construction uses m structured odd-degree polynomials with explicit parity constraints (even base point g , odd secret components) and a cyclic wiring that reuses the secret vector under rotation. For the initial proof of concept implementation in Python see [Gli26]. Extensive numerical validation (2D parameter sweeps over base width w , 2-adic valuation $v_2(g)$, rounds R , and polynomial degree) shows that:

- the effective image size of both the public key and the shared secret shrinks as a power of two with $v_2(g)$;
- for all safe parameter regions ($w \geq 40$ or $v_2(g) \geq 3$) the sets of public keys and shared secrets are *completely disjoint*;
- odd round counts often yield approximately twice the image size of even counts (“lucky rounds”).

These properties provide strong evidence that CQA realizes a large functional graph on its effective image while keeping the public-key orbit and the shared-secret orbit in algebraically separated regions — exactly the behaviour required for a secure commutative-action KEX. We describe the construction, connect it to commutative-action thinking in isogeny cryptography, prove a 1-dimensional collapse attack when g is odd, and list concrete open problems.

1 Commutative Quasigroup Action

1.1 General m -ary cyclic quasigroup action

Fix an arity $m \geq 3$ and a modulus 2^w (with w a security parameter). A *family* $\mathcal{F} = (g, (P_1, \dots, P_m))$ consists of a public base point $g \in \mathbb{Z}/2^w\mathbb{Z}$ and m structured odd-degree polynomials $P_1, \dots, P_m$ over $\mathbb{Z}/2^w\mathbb{Z}$. Each polynomial is required to satisfy explicit parity constraints that enforce the 2-adic regime:

- the constant term of g is even ($g = 2s$);
- all secret components (the vector $K = (k_1, \dots, k_{m-1})$) are odd ($k_i = 2s_i + 1$);
- the polynomials P_j have odd degree and coefficients compatible with the above parity (leading coefficient even for the g -slot, odd otherwise).

These constraints are not arbitrary; they are the minimal conditions under which the cyclic action commutes in the 2-adic topology (see Section 1.1).

The local law. The elementary operation is the multilinear-like product

$$f(x_1, \dots, x_m) := \prod_{i=1}^m P_i(x_i) \pmod{2^w}.$$

Because each P_i has odd degree, f is a permutation polynomial on the units of $\mathbb{Z}_{2^w}$ when the arguments are units; the even/odd parity constraints extend this behaviour to the full ring in a controlled way.

Cyclic wiring (macro-step). Let $K = (k_1, \dots, k_{m-1})$ be the secret vector and let x be the current state. A single *macro-step* $M_K(x)$ is defined by the following cyclic insertion procedure (exactly as implemented in the reference Python proof-of-concept):

1. Start with the length- m argument vector $\mathbf{a}^{(0)} = (k_1, \dots, k_{m-1}, x)$.
2. For $i = 0, \dots, m-2$:
 - Compute $y_{i+1} = f(\mathbf{a}^{(i)})$.
 - Rotate the secret part of $\mathbf{a}^{(i)}$ one position to the left and insert y_{i+1} into the vacated slot, obtaining $\mathbf{a}^{(i+1)}$.
3. Return y_{m-1} as the output of the macro-step.

The full action is the R -fold iteration:

$$\text{Act}(K, x) := M_K^R(x).$$

For $m = 3$ the procedure specialises exactly to the three-micro-step wiring given in the earlier draft:

$$y_1 = f(a, b, x), \quad y_2 = f(b, y_1, a), \quad y_3 = f(y_2, a, b), \quad M_{(a,b)}(x) = y_3.$$

1.2 Design rationale: necessity of an even base point g .

The parity constraints are not merely implementation details; they are required to avoid a fatal one-dimensional collapse. Consider the homogeneous specialisation in which all constant terms of the P_j vanish and g is odd (i.e., a unit in $\mathbb{Z}_{2^w}$). Then the entire macro-step collapses to pure scalar multiplication:

$$M_K(x) = \mu(K) \cdot x, \quad \mu(K) \in \mathbb{Z}/2^w\mathbb{Z}.$$

(The scalar $\mu(K)$ is a monomial in the components of K whose precise form depends on m and the degrees of the P_j .) Consequently the public key becomes

$$A \cdot g = \mu(A) \cdot g.$$

Because g is invertible, an adversary who sees $A \cdot g$ can immediately recover the full action scalar:

$$\mu(A) = (A \cdot g) \cdot g^{-1}.$$

He can then compute the shared secret without ever learning A or B :

$$\text{ss} = \mu(A) \cdot (B \cdot g) = \mu(A) \cdot \mu(B) \cdot g.$$

This is precisely the “hidden homogeneous space” attack that renders any one-dimensional commutative action insecure. The requirement that g be even (together with odd secret components and odd-degree polynomials) prevents the homogeneous collapse and forces the action to explore a genuinely higher-dimensional subset of the ring. Extensive numerical experiments confirm that, once $v_2(g)$ is large enough relative to w and R , the resulting functional graph on the effective image is large and the public-key and shared-secret orbits remain disjoint — the algebraic signature of a secure commutative action.

2 KEX protocol (CSIDH-like commutative action).

Public parameters are a family $\mathcal{F} = (g, (P_1, \dots, P_m))$ together with the round count R . Each party samples a secret vector (A for Alice, B for Bob) whose components satisfy the odd-parity constraint and publishes

$$A \cdot g := \text{Act}(A, g), \quad B \cdot g := \text{Act}(B, g).$$

The shared secret is computed as

$$\text{ss} := \text{Act}(A, B \cdot g) = \text{Act}(B, A \cdot g),$$

where equality holds throughout the safe 2-adic regime (empirically verified for thousands of random instances across $w \in \{32, 36, 40, 44, 48\}$, $v_2(g) \in \{1, \dots, 18\}$, $R \in \{2, \dots, 16\}$, and polynomial degrees 3 and 9). A standard KDF (e.g., HKDF-SHA-256) is applied to ss to derive session keys.

2.1 Communication sizes and effective entropy.

A public key is a single element of $\mathbb{Z}/2^w\mathbb{Z}$ and therefore costs w bits ($\lceil w/8 \rceil$ bytes). The KEX transcript consists of two public keys plus optional KDF context. A secret key is a vector of $m-1$ odd elements mod 2^w ; a naïve encoding costs $(m-1)w$ bits. In the 2-adic regime the distribution of each secret component has a frozen low-order suffix whose length grows with $v_2(g)$ and R . Consequently the *effective* entropy per component is approximately

$$\text{eff_bits} \approx w - v_2(g) - c \cdot R$$

for a small constant c that depends on the polynomial degree (empirically $c \approx 1$ for degree 3 and slightly larger for degree 9). The 2D sweep data show that safe parameter choices (those that keep Apub and ssA disjoint) still leave $\text{eff_bits} \geq w/2$ for $w \geq 40$ and modest R , which is sufficient for conservative λ -bit security against generic collision search.

2.2 Empirical validation (2D parameter sweeps).

We performed exhaustive adaptive 2D sweeps over the rectangle

$$w \in \{32, 36, 40, 44, 48\}, \quad v_2(g) \in \{1, \dots, 18\}, \quad R \in \{2, \dots, 16\}$$

for both degree-3 and degree-9 odd polynomials (more than 2700 distinct cells, each sampled until saturation or a 10^7 sample cap). The results reveal three robust statistical laws:

1. **Power-of-two image sizes.** Both $|\text{image}(\text{Apub})|$ and $|\text{image}(\text{ssA})|$ are always exact powers of two. The size halves (to within ± 1) with every increment of $v_2(g)$.
2. **“Lucky rounds”.** For any fixed $(w, v_2(g))$, an odd number of rounds yields an image approximately twice as large as the nearest even count. This periodic mixing gain is reproducible across all tested widths and valuations.
3. **Strong set separation.** For every cell with $w \geq 40$ or $v_2(g) \geq 3$ we observed

$$|\text{Apub} \cap \text{ssA}| = 0$$

(perfect algebraic separation). The only exceptions form a narrow “bad zone” confined to $(w = 32, v_2(g) \leq 2)$ and, for higher-degree polynomials, a slightly larger but still tiny region around $(w = 36, v_2(g) = 2)$. In all bad-zone cells the overlap is exactly 100% ($\text{Apub} = \text{ssA}$); outside it the overlap is exactly 0%.

These laws confirm that the construction realises a large functional graph on its effective image while automatically placing the public-key orbit and the shared-secret orbit in disjoint subsets — the precise algebraic property needed for a secure commutative-action KEX.

3 Open research questions.

1. **Hardness assumptions.** The natural computational problems are:
 - **Vectorization / key recovery.** Given $\mathcal{F}$, g and $y = \text{Act}(K, g)$, recover a vector K' with $\text{Act}(K', g) = y$.
 - **Shared-secret computation.** Given g , $A \cdot g$ and $B \cdot g$, compute $\text{ss} = \text{Act}(A, B \cdot g)$ without recovering A or B .
 - **Distinguishing.** Distinguish the distribution of ss from uniform on the observed image (or uniform modulo the frozen suffix).

The cyclic wiring together with the nonlinear odd-degree polynomials and the even- g constraint make reduction to a single “missing quasigroup argument” impossible; the 1D collapse attack is provably blocked. Whether these problems are hard in the 2-adic regime remains the central open question.

2. **Characterisation of the commutative 2-adic subfamily.** Why exactly does the parity-constrained family commute? Can one give an algebraic characterisation of the maximal commutative subfamily, or an efficient algorithm that decides membership?
3. **Parameter selection (conservative profiles).** Combining the image-size law with the separation guarantee, we propose the following draft tiers (all with $m = 3$, degree 3, $R = 2$ or 3):
 - **128-bit tier:** $w = 512$ ($v_2(g) = 4..6$), public key 64 bytes, KEX transcript 128 bytes.
 - **192-bit tier:** $w = 768$ ($v_2(g) = 5..7$), public key 96 bytes, KEX 192 bytes.
 - **256-bit tier:** $w = 1024$ ($v_2(g) = 6..8$), public key 128 bytes, KEX 256 bytes.

These choices keep the construction comfortably inside the perfect-separation region while retaining $\text{eff_bits} \geq 2\lambda$.

4. **Security models (CPA/CCA and AKE).** The current Python proof-of-concept validates commutativity, serialization and KDF usage in the passive setting. Upgrading to active security (IND-CCA KEM, AKE) is an open engineering and proof challenge.
5. **Attacks and leakage.** Identify what structure is necessarily leaked by the 2-adic regime (frozen suffix, image-size law) and how KDF usage, masking, or protocol-level choices mitigate it.
6. **Signatures and identification.** Explore whether CQA admits identification/signature analogues of group-action signatures without breaking commutativity.

4 Symbolic affine commutator calculations

The following symbolic derivation shows how a 3-ary affine model becomes a unary affine self-map and why affine commutativity is too weak to be cryptographically interesting. (The same expansion works verbatim for general m with only notational changes.)

We work in the m -ary cyclic wiring with

$$f(u_1, \dots, u_m) = P_1(u_1) \cdots P_m(u_m)$$

and affine slot polynomials

$$P_j(t) = c_j + d_j t, \quad j = 1, \dots, m.$$

Proposition 4.1 (Affine m -ary macro-step). *Under the affine truncation the full macro-step is affine in the state variable:*

$$M_K(x) = \alpha(K) + \beta(K)x.$$

The explicit formulae for α and β are obtained by tracking the constant term and the coefficient of x through the $m - 1$ micro-steps; they are multilinear in the secret components and involve only the constants c_j and the slopes d_j .

Proposition 4.2 (Commutator obstruction in the affine regime). *Let A and B be two secret vectors and write $M_A(x) = \alpha_A + \beta_A x$, $M_B(x) = \alpha_B + \beta_B x$. Then*

$$M_A(M_B(x)) - M_B(M_A(x)) = \alpha_A(1 - \beta_B) - \alpha_B(1 - \beta_A).$$

In particular, affine-slot multilinearity alone does not imply useful commutativity.

Proposition 4.3 (Homogeneous specialization). *If all constants vanish ($c_j \equiv 0$) then the macro-step collapses to pure scalar multiplication*

$$M_K(x) = \left(\prod_{j=1}^m d_j \right) \left(\prod_{i=1}^{m-1} k_i^d \right) x$$

for a monomial exponent d that depends only on the polynomial degrees. Two such scalar maps commute automatically, but the construction is exactly the one-dimensional case shown to be insecure when g is odd (Section 1.2).

5 Conclusion

The generic m -ary construction, the explicit parity constraints, the even- g requirement, and the extensive numerical validation together give a concrete, implementable candidate for a post-quantum commutative-action key exchange whose security rests on the hardness of vectorization and shared-secret computation in a genuinely high-dimensional 2-adic regime. The only remaining task is to turn the empirical evidence into a rigorous hardness proof — or to discover an attack that exploits the observed image-size law.

References

- [CLM⁺18] Wouter Castryck, Tanja Lange, Chloe Martindale, Lorenz Panny, and Joost Renes. Csidh: An efficient post-quantum commutative group action. In *Advances in Cryptology – ASIACRYPT 2018*, volume 11274 of *Lecture Notes in Computer Science*, pages 395–427. Springer, 2018. Also available as IACR ePrint 2018/383.
- [DF17] Luca De Feo. Mathematics of isogeny based cryptography. *CoRR*, abs/1711.04062, 2017.
- [Gli26] Danilo Gligoroski. Commutative-quasigroup-actions (cqa) proof-of-concept. <https://github.com/danilo-gligoroski/Commutative-Quasigroup-Actions>, 2026. GitHub repository, accessed 2026-05-02.
- [MŠG10] S. Markovski, Z. Šunić, and D. Gligoroski. Polynomial functions on the units of $\mathbb{Z}_{2^n}$. *Quasigroups and Related Systems*, 23(1):59–82, 2010.